YaP Link — native proxy (Velocity-class, phased)

Product decision: YaP Link is a **first-party Netty proxy** (`com.yapcore.link.*`), evolved from `yap-first-party/link/native/`. It is **not** a Velocity fork.

Status: Phases 0–6 in native Link (`0.6.0-phase6`).

See also: `yap-first-party/link/api/` · `yap-first-party/link/protocol/` · `yap-first-party/link/plugins/`

Velocity parity matrix

Feature	Velocity	YaP Link	Phase
Modern player-info forwarding	✓	✓	**0** ✓
Multi-backend + try order	✓	✓ `link.properties` / `link.toml`	**0** ✓
Online / offline login	✓	✓	**0** ✓
Compression bridge	✓	✓	**0** ✓
`/server` + Transfer reconnect	✓	✓	**0** ✓
`link.properties` + `link.toml`	—	✓	**0** ✓
Ping **passthrough**	✓	✓ cached backend probe	**1** ✓
Forced hosts	✓	✓ `forced-host.<host>=server`	**1** ✓
Backend health + try failover	✓	✓ `BackendMonitor`	**1** ✓
Connect / login / read timeouts	✓	✓	**1** ✓
Play-phase system chat	✓	✓ `PlayChat`	**1** ✓
Aggregate player count in ping	✓	✓ `aggregate-player-count`	**2** ✓
Cross-server chat relay	plugins	✓ `ChatRelay` + `say` console	**2** ✓
Config reload	✓	✓ console `reload`	**2** ✓
YaP Link plugin API	Velocity API	✓ `yap-link-api`	**3** ✓
chat-bridge / mod-sync / server-selector	Velocity plugins	✓ `yap-first-party/link/plugins/`	**3** ✓

Feature	Velocity	YaP Link	Phase
Bedrock UDP edge + per-backend routing	Geyser on proxy	✓ `BedrockUdpForwarder`	**4** ✓
Floodgate key forwarding	Floodgate on proxy	✓ `floodgate-key.pem`	**4** ✓
Metrics hooks	partial	✓ `LinkMetrics` + `/metrics`	**5** ✓ / **Edge** ✓
Connect / handshake rate limit	plugins	✓ per-IP (default ON)	**Edge** ✓
`link-embed` in YaPcore	—	✓ `link-embed=true`	**5** ✓
Release bundle	—	✓ `yap-link.jar` in `assembleRelease`	**5** ✓
Velocity fork retired	—	✓ archived	**5** ✓
Play plugin-message relay (`yap:chat`)	plugins	✓ `PluginMessagePackets`	**6** ✓
Two-backend probe + chat relay	—	✓	**6** ✓
Bedrock UDP forward to backend	—	✓	**6** ✓

Phase 6 — Hardening

Task	Gate
Play-phase `PluginMessageEvent` for registered channels	Unit tests + chat-bridge relay
Multi-backend probe + `say` chat relay	
Bedrock UDP datagram forward to backend	

Config

Home: `link-data/` (or `--home`). Prefer **`link.toml`** or **`link.properties`**.

```
bind=0.0.0.0:25565
motd=YaP Link
servers.lobby=127.0.0.1:25566
servers.survival=127.0.0.1:25567
servers.lobby.bedrock=127.0.0.1:19132
try=lobby,survival
```

```
ping-passthrough=true
plugins-enabled=false
# First run: ./scripts/start-yap-link.sh seeds plugins-enabled=true + plugin jars
floodgate-key-file=floodgate-key.pem
bedrock-enabled=false
bedrock-bind=0.0.0.0:19132
bedrock-backend=127.0.0.1:19132
```

Plugins load from `link-data/plugins/*.jar` with `link-plugin.json` descriptors.

Build & run

```
gradle :yap-link-native:shadowJar
gradle :yap-link-plugin-chat-bridge:installIntoLinkPlugins   # optional
./scripts/start-yap-link.sh
```

Embedded Link (dev / single-box): `config/server.properties`:

```
link-embed=true
link-embed-home=link-data
```

Requires `yap-link.jar` on the classpath alongside `yapcore.jar`.

Release: `gradle assembleRelease` ships `yapcore.jar`, `yap-link.jar`, and `link-data/plugins/`.

Console commands: `help` · `reload` · `list` · `servers` · `say <msg>` · `stop`

Architecture

```
Players (JE TCP / optional BE UDP)
        │
        ▼
┌────────────────────────────────────────┐
│  YaP Link (native JVM) :25565          │
│  yap-link.jar · link.properties        │
│  LinkPluginManager · FloodgateForwarder│
│  BedrockUdpForwarder (per-backend)     │
└────────────────┬───────────────────────┘
                │ optional velocity:player_info
                ▼
┌────────────────────────────────────────┐
```

```
│  YaPcore Via edge :25566               │
└────────────────────────────────────────┘
                 │
                 ▼
┌────────────────────────────────────────┐
│  YaP-Folia game :25567                 │
│  (folia-jar-source=build)              │
└────────────────────────────────────────┘
```

Shared wire code: `yap-first-party/link/protocol/` (`McCodec`, `McFrameCodec` inbound, `McOutboundPacketEncoder` outbound zlib+frame, `McCompressionCodec` inbound decompress + `wrapOutbound`, modern forwarding, Floodgate).

Outbound rule: one handler for zlib+length (`McOutboundPacketEncoder`). Stacking compress `MessageToMessageEncoder` + frame encoder caused vanilla `CorruptedFrameException: Frame length cannot be zero` after Set Compression.

Login Success: if the backend never sends `velocity:player_info` (Folia velocity disabled), Link still bridges — forwarding is optional unless the backend requests it.

Jar the GUI starts: repo-root `yap-link.jar` (preferred) or `yap-first-party/link/native/build/libs/yap-link.jar`. Always republish/copy after protocol fixes so operators are not on a stale root jar.

Bedrock / Geyser

Bedrock UDP routing at Link is **shipped** (`BedrockUdpForwarder`) but **off by default** (`bedrock-enabled=false`). For single-box crossplay, use chassis dual-stack on the **YaP-Folia** backend. For BE-heavy multi-backend networks, enable Link Bedrock bind or use stock Velocity as a stand-in. See YAP_LINK.md.

Agent handoff (Phases 0–2 ↔ 3–5)

For the 0–2 agent:

Rule	Detail
Plugins default OFF in code	`LinkConfig.applyDefaults()` → `plugins-enabled=false`. Unit tests and bare `LinkConfig.load()` stay plugin-free until config opts in.
First run / release turns plugins ON	`./scripts/start-yap-link.sh` seeds `link.properties` with `plugins-enabled=true`, builds `yap-link-plugin-*` jars into `link-data/plugins/`. `assembleRelease` should mirror that seed.

Rule	Detail
Bedrock ctor	**`new BedrockUdpForwarder(LinkConfig)`** only — not the old 4-arg host/port stub.
Plain chat	Built-in **`ChatRelay`** (Phase 2) — serverbound play chat + console `say`.
Backend `yap:chat`	**`yap-link-plugin-chat-bridge`** when backends send on channel `yap:chat` and `plugins-enabled=true` (`PluginMessageEvent`). Do not duplicate in `ClientSession`.

`LinkServer` owns: `LinkPluginManager`, `FloodgateForwarder`, `BedrockUdpForwarder(config)`, plus Phase 0–2 `BackendMonitor` / `ChatRelay` / `PlayerHub`.

Full play-phase plugin-message wire sniffing (all channels) is **optional future work** — not required for Phase 0–2 or for `yap:chat` when chat-bridge is loaded.

Boundary: Phase 0–2 stops at `ChatRelay`, `BackendMonitor`, `PlayChat`, console `reload`. Do not grow `ClientSession` into a second plugin platform.

Related

- EDGE_RATE_LIMIT.md · EDGE_HARDEN.md — rate limits, Prometheus, public edge harden
- YAP_LINK.md — operator entry
- VELOCITY.md — Folia backend forwarding
- YAPCORE_WHITEPAPER.md — chassis vs Link process boundaries